Divisão e Conquista

Prof. Luis A. A. Meira

1 de setembro de 2025

Resumo

Nesta aula veremos uma técnica de desenvolvimento de algoritmos chamada divisão e conquista.

Considere o algoritmo genérico:

Algorithm 1: DivisaoConquista(x)

Input: A instância x

Output: Solução y do problema em questão para x

```
if x é suficientemente pequeno then
    // Solucao(x) algoritmo para pequenas instâncias
    return Solucao(x);
else
    // divisão
    Decomponha x em instâncias menores x_1, x_2, ..., x_k;
    for i ← 1 até k do
        y_i ← DivisaoConquista(x_i);
    // conquista
    Combine as soluções y_i para obter a solução y de x;
    return y;
```

Considere então o problema de calcular a^n para a real e n inteiro positivo. A maneira tradicional de calcular é por indução fraca, ou seja, para $n-1$.

Problema: Calcular a^n, para todo real a e inteiro $n \geq 0$.

Primeira solução, por indução fraca:

- **Caso base:** $n = 0$; $\quad a^0 = 1$.
- **Hipótese de indução:** *Suponha que, para qualquer inteiro $n > 0$ e real a, sei calcular a^{n-1}.*
- **Passo da indução:** Queremos provar que conseguimos calcular a^n, para $n > 0$. Por hipótese de indução, sei calcular a^{n-1}. Então, calculo a^n multiplicando a^{n-1} por a.

O código abaixo implementa esta indução em duas versões, uma recursiva e outra iterativa:

```c
#include <stdio.h>
double pow1(double a, int n) {
    double r = 1.0;
    for (int i = 0; i < n; i++) {
        r *= a;
    }
    return r;
}
double pow2(double a, int n) {
    if(n<=0)return 1.0;
    return a*pow2(a,n-1);
}

int main(void){
    printf("%.1lf\n",pow1(2.0,10));
    printf("%.1lf\n",pow2(2.0,10));
}
```

A versão *pow1* consiste em um *for* com n passos e é, portanto, $\Theta(n)$.

Seja $T(n)$ o tempo de execução da função `pow2(a,n)`.

- Caso base ($n = 0$): a função retorna imediatamente.

$$T(0) = c, \quad c > 0.$$

- Caso geral ($n > 0$): há uma multiplicação (custo constante c) e uma chamada recursiva com $n - 1$.

$$T(n) = T(n-1) + c.$$

Expandindo a recorrência:

$$\begin{aligned} T(n) &= T(n-1) + c && (1)\\ &= T(n-2) + 2c && (2)\\ &= \dots && (3)\\ &= T(0) + nc. && (4)\\ & && (5) \end{aligned}$$

Como $T(0) = c$, obtemos:

$$T(n) = c + nc = c(n+1).$$

Portanto, assintoticamente:

$$T(n) \in \Theta(n).$$

Segunda solução, por indução forte:

- **Caso base:** $n = 0; \quad a^0 = 1.$
- **Hipótese de indução:** *Suponha que, para qualquer inteiro $n > 0$ e real a, sei calcular a^k, para todo $k < n$.*
- **Passo da indução:** Queremos provar que conseguimos calcular a^n, para $n > 0$. Por hipótese de indução sei calcular $a^{\lfloor n/2 \rfloor}$. Então, calculo a^n da seguinte forma:

$$a^n = \begin{cases} \left(a^{\lfloor n/2 \rfloor}\right)^2, \text{se } n \text{ par;} \\ a \cdot \left(a^{\lfloor n/2 \rfloor}\right)^2, \text{se } n \text{ ímpar.} \end{cases}$$

Veja duas implementações deste código:

```c
#include <stdio.h>
double pow3(double a, int n) {
    if(n<=0)return 1.0;
    double aux = pow3(a,n/2)*pow3(a,n/2);
    if(n%2==0){
        return aux;
    }else{
        return aux*a;
    }
}
double pow4(double a, int n) {
    if(n<=0)return 1.0;
    double aux = pow4(a,n/2);
    if(n%2==0){
        return aux*aux;
    }else{
        return aux*aux*a;
    }
}
int main(void){
    printf("%.1lf\n",pow3(2.0,10));
    printf("%.1lf\n",pow4(2.0,10));
}
```

A implementação *pow3* faz duas chamadas recursivas, o que gera a relação de recorrencia:

$$T(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + c, \text{se } n > 0 \\ c, \text{se } n = 0 \end{cases}$$

Ainda não aprenderemos a resolver este tipo de relação de recorrência, mas a resposta é $T(n) = \Theta(n)$ para *pow*3. Já para o algoritmos *pow4* só é realizada uma chamada recursiva. Assim, o tempo será descrito pela seguinte relação de recorrência:

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + c, \text{se } n > 0 \\ c, \text{se } n = 0 \end{cases}$$

Futuramente iremos ver como resolver tal relação de recorrência, mas ela gera uma complexidade $T(n) = \Theta(\lg n)$

Vamos a um novo problema: Máximo e Mínimo

Problema: Dado um conjunto S de $n \geq 2$ números reais, determinar o maior e o menor elemento de S. Tal problema é definido como $minMax(S, n)$.

Segue a indução matemática:

- **Caso base:** $n = 2$; retorne S_0, S_1 se $S_0 \leq S_1$ ou S_1, S_0 caso contrário.
- **Hipótese de indução:** Suponha que é possível encontrar $minMax(S, n-1)$.
- **Passo da indução:**

  calcule $min, max \leftarrow minMax(S, n-1)$

  Se $S[n-1] < min$ então $min \leftarrow S[n-1]$

  Se $S[n-1] > max$ então $max \leftarrow S[n-1]$

  devolva min, max

Veja o código abaixo:

```
def minMax1(S, n):
    if n==2:
        if S[1] < S[2]:
            return S[1],S[2]
        else:
            return S[2],S[1]
    mini,maxi=minMax1(S,n-1)
    if S[n-1]<mini:
        mini=S[n-1]
    if S[n - 1] > maxi:
        maxi = S[n - 1]
    return mini,maxi

```

```
S = [1,2,3,4,5,6,-1]
print(minMax1(S,len(S)))
```

A relação de recorrencia que rege este algoritmo é $T(n) = T(n-1)+c$ que resulta em $T(n) = \Theta(n)$. Um algoritmo, para encontrar o mínimo e o máximo precisa olhar para todas as posições do vetor. Se uma posição não for analisada, a solução está potencialmente errada. Logo, é impossível resolver este problema em menos de $\Theta(n)$.

Agora, vamos analisar o número de comparações que o algoritmo faz. Seja $T(n)$ o número de comparações realizadas pelo algoritmos. Temos que $T(2) = 1$. Se o vetor cair no caso base, ele faz apenas uma comparação. Para valores maiores de n temos $T(n) = 2 + T(n-1)$. Pelo método da substituição, o número de comprações será $T(n) = 2n + 3$. Será que conseguimos reduzir o número de comparações realizado pelo algoritmo?

- Será que um algoritmo de divisão e conquista seria melhor?
- Um possível algoritmo de divisão e conquista seria:
  - Divida S em dois subconjuntos de mesmo tamanho S_1 e S_2 e solucione os subproblemas.
  - O máximo de S é o máximo dos máximos de S_1 e S_2 e o mínimo de S é o mínimo dos mínimos de S_1 e S_2.
- Qual o número de comparações $T(n)$ efetuado por este algoritmo?

$$T(n) = \begin{cases} 1, & n = 2 \\ T\left(\lfloor \frac{n}{2} \rfloor\right) + T\left(\lceil \frac{n}{2} \rceil\right) + 2, & n > 2 \end{cases}$$

- Supondo que n é uma potência de 2, temos:

$$T(n) = \begin{cases} 1, & n = 2 \\ 2T\left(\frac{n}{2}\right) + 2, & n > 2 \end{cases}$$

- Neste caso, podemos provar que

$$T(n) = \tfrac{3}{2}n - 2$$

  usando o método indução.

- **Afirmação:** $T(n) = \frac{3}{2}n - 2$.
- **Caso Base:** $T(2) = 1 = 3 - 2$.
- **Hipótese de Indução:** Suponha, para $n' = n/2$, que

$$T(n') = \tfrac{3}{2}n' - 2.$$

- **Passo de Indução:** Queremos provar

$$T(n) = \tfrac{3}{2}n - 2.$$

$$
\begin{aligned}
T(n) &= 2T\left(\tfrac{n}{2}\right) + 2 \\
&= 2\left(\tfrac{3}{4}n - 2\right) + 2 \quad \text{(hipótese)} \\
&= \tfrac{3}{2}n - 2.
\end{aligned}
$$

- Assintoticamente, os dois algoritmos para este problema são equivalentes, ambos $\Theta(n)$.
- No entanto, o algoritmo de divisão e conquista permite que menos comparações sejam feitas. A estrutura hierárquica de comparações no retorno da recursão evita comparações desnecessárias.